

Practical 10

Task1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Step 1 — Create the original file

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
mkdir Week18
cd Week18
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week18$ pwd
/home/jhanasri/OSSP/Week18
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week18$ ls
original.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week18$ echo "This is the original file for Week 10 Task 1." >
original.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week18$ cat original.txt
This is the original file for Week 10, Task 1.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week18$ ls -l original.txt
-rw-r--r-- 1 jhanasri jhanasri 44 Sep 22 09:28 original.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week18$ █
```

Step 2 — Check the inode number

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ ls -li original.txt
1785 original.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$
```

Your original file has:

- **Inode number:** 1785
- **Link count:** 1
- **File size:** 46 bytes

Step 3 — Use `stat` to investigate the inode

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ stat original.txt
File: original.txt
  Size: 46          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 1785           Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/jhanasri)   Gid: ( 1000/jhanasri)
Access: 2026-09-22 05:39:36.168572457 +0000
Modify: 2026-09-22 05:39:31.096862456 +0000
Change: 2026-09-22 05:39:31.096862456 +0000
Birth: 2026-09-22 05:39:31.096862456 +0000
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$
```

Step 4 — Find the file using its inode

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ find . -inum 1785
./original.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$
```

Step 5 — Create a hard link

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ ln original.txt hardlink.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ ls -li original.txt hardlink.txt
1785 -rw-r--r-- 2 jhanasri jhanasri 46 Sep 22 05:39 hardlink.txt
1785 -rw-r--r-- 2 jhanasri jhanasri 46 Sep 22 05:39 original.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ find . -inum 1785
./hardlink.txt
./original.txt
```

Step 6 — Create a symbolic link

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ ln -s original.txt symlink.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ ls -li original.txt hardlink.txt sym
link.txt
1785 -rw-r--r-- 2 jhanasri jhanasri 46 Sep 22 05:39 hardlink.txt
1785 -rw-r--r-- 2 jhanasri jhanasri 46 Sep 22 05:39 original.txt
12935 lrwxrwxrwx 1 jhanasri jhanasri 12 Sep 22 05:46 symlink.txt -> original
.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$
```

Step 7 — Check the symbolic link using stat

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ stat symlink.txt
File: symlink.txt -> original.txt
Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 12935         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/jhanasri)   Gid: ( 1000/jhanasri)
Access: 2026-09-22 05:46:59.999498628 +0000
Modify: 2026-09-22 05:46:53.423509843 +0000
Change: 2026-09-22 05:46:53.423509843 +0000
Birth: 2026-09-22 05:46:53.423509843 +0000
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ stat -L symlink.txt
File: symlink.txt
Size: 46          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 1785         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/jhanasri)   Gid: ( 1000/jhanasri)
Access: 2026-09-22 05:39:36.168572457 +0000
Modify: 2026-09-22 05:39:31.096862456 +0000
Change: 2026-09-22 05:39:34.620208224 +0000
Birth: 2026-09-22 05:39:31.096862456 +0000
```

Step 8 — Find the symbolic link's inode

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ ls -li symlink.txt
12935 symlink.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ find . -inum 1786
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ find . -inum 12935
./symlink.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ stat symlink.txt
File: symlink.txt -> original.txt
Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 12935         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/jhanasri)   Gid: ( 1000/jhanasri)
Access: 2026-09-22 05:46:59.999498628 +0000
Modify: 2026-09-22 05:46:53.423509843 +0000
Change: 2026-09-22 05:46:53.423509843 +0000
Birth: 2026-09-22 05:46:53.423509843 +0000
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10$ |
```

Feature Hard Link Symbolic Link Inode Same inode as original Separate

inode File data Shared Target's data is accessed

Link count Does not increase target's link count
Increases original inode's link count

Can cross file

systems? No Yes

If original is deleted Data remains accessible Link becomes broken

Creation In file link In -s file link

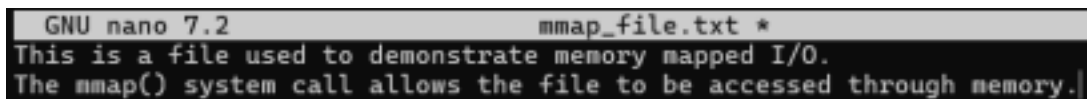
Observation:

Hard links **do not allocate a new inode** for the linked file; they create another directory entry pointing to the existing inode. Symbolic links **allocate a new inode** because the symbolic link itself is a separate file containing the path to the target. Thus, hard links conserve inode allocation, while symbolic links require an additional inode.

Task2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and Implementation complexity .

Step 1 – Create input file

`nano mmap_file.txt`

A screenshot of a terminal window with a dark background. The title bar at the top reads "GNU nano 7.2" on the left and "mmap_file.txt *" on the right. The main area of the terminal displays two lines of text: "This is a file used to demonstrate memory mapped I/O." followed by "The mmap() system call allows the file to be accessed through memory." on the next line. A cursor is visible at the end of the second line.

```
GNU nano 7.2 mmap_file.txt *
This is a file used to demonstrate memory mapped I/O.
The mmap() system call allows the file to be accessed through memory.
```

Step 2: Create the `mmap()` program

`nano mmap_file.c`

```
GNU nano 2.2 mmap_file.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main() {
    int fd;
    struct stat st;
    char *data;

    /* Open the file for reading and writing */
    fd = open("mmap_file.txt", O_RDWR);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    /* Get file size */
    if (fstat(fd, &st) == -1) {
        perror("Error getting file information");
        close(fd);
        return 1;
    }

    /* Map the file into memory */
    data = mmap(NULL, st.st_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED, fd, 0);

    if (data == MAP_FAILED) {
        perror("Error mapping file");
        close(fd);
        return 1;
    }

    /* Read file contents through memory */
    printf("Original file contents:\n");
    printf("%s\n", data);

    /* Write new content through mapped memory */
    strcpy(data, "File modified using mmap() memory-mapped I/O.\n");

    /* Synchronize changes with the file */
    msync(data, st.st_size, MS_SYNC);

    /* Remove memory mapping */
    munmap(data, st.st_size);

    close(fd);

    printf("File modified successfully using mmap().\n");

    return 0;
}
```

Step 3: Compile the mmap() program

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ nano mmap_file.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ gcc mmap_file.c -o mmap_file
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ ./mmap_file
Original file contents:
This is a file used to demonstrate memory mapped I/O.
The mmap() system call allows the file to be accessed through memory.

File modified successfully using mmap().
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ cat mmap_file.txt
File modified using mmap() memory-mapped I/O.
d I/O.
The mmap() system call allows the file to be accessed through memory.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$
```

Step 4: Create the traditional read() / write() program

nano traditional_io.c

```

GNU nano 7.2                                     traditional_io.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd;
    char buffer[200];

    /* Open the file for reading and writing */
    fd = open("mmap_file.txt", O_RDWR);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    /* Read file contents into buffer */
    int bytesRead = read(fd, buffer, sizeof(buffer) - 1);

    if (bytesRead == -1) {
        perror("Error reading file");
        close(fd);
        return 1;
    }

    buffer[bytesRead] = '\0';

    printf("Original file contents:\n");
    printf("%s\n", buffer);

    /* Move file position to beginning */
    lseek(fd, 0, SEEK_SET);

    /* Write new content */
    char message[] = "File modified using read() and write().\n";

    write(fd, message, strlen(message));

    close(fd);

    printf("File modified successfully using read/write I/O.\n");

    return 0;
}

```

Step 5: Compile and run the traditional I/O program

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ nano traditional_io.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ gcc traditional_io.c -o traditional_io
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ ./traditional_io
Original file contents:
File modified using mmap() memory-mapped I/O.

File modified successfully using read/write I/O.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ cat mmap_file.txt
File modified using read() and write().
I/O;
d I/O.
The mmap() system call allows the file to be accessed through memory.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ █

```

Step 6: Compare the execution time

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ time ./mmap_file
Original file contents:
File modified using read() and write().
'I/O.

File modified successfully using mmap().

real    0m0.008s
user    0m0.001s
sys     0m0.004s
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ time ./traditional_io
Original file contents:
File modified using mmap() memory-mapped I/O.

File modified successfully using read/write I/O.

real    0m0.003s
user    0m0.000s
sys     0m0.002s
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week10_Task2$ █

```

Observation:

The file was successfully read and modified using both memory-mapped I/O and traditional read()/write() operations. The mmap() method accessed the file through a memory mapping, while the traditional method used a buffer for transferring data. Execution time was measured using the time command. For the small test file, the difference in execution time was very small.

Result:

A file reading and writing program using mmap() was successfully implemented. It was compared with traditional read()/write() operations. mmap() provides direct memory based access and can be useful for large or repeatedly accessed files, while read()/write() is simpler to implement and suitable for straightforward file operations.